,

Assignment Six
Hardware Lab: CS224

Shelke Durgesh Balkrishna: 230101093

Kavya Kumar Agrawal: 230101053

Parth Sunil Aher: 230101072

Dhruv Pansuriya: 230101071

1st April 2025

1 Problem Statement

The given function is defined as:

```
module MaxMin(
input [3:0] wAddress, // address where dataIn to be written
input [4:0] dataIn, // data to be written at wAddress location
input dataValid, // will be on when we want to write input data to the memory
input clk, // clock signal
output reg outValid, // is 1 when processing is done and valid data is showing at output
output reg [3:0] outAddress, // address corresponding to dataOut
output reg [4:0] dataOut // output data corresponding to outAddress location
);

Memory operations and max/min finding algorithm
Read data into memory when dataValid is 1
Process data to find maximum and minimum values
Output results one by one with outValid set to 1
```

2 Logic behind the circuit obtained

The circuit design is based on fundamental digital logic principles, leveraging combinational and sequential logic elements to achieve the desired functionality.

2.1 Combinational Logic

The combinational logic in this circuit includes multiplexers, comparators, and arithmetic units. These components perform operations such as data selection, comparison, and addition without relying on memory or past states. For example:

- Multiplexers are used to route different inputs to specific outputs based on control signals.
- Comparators determine the maximum and minimum values by comparing data stored in registers.
- Arithmetic units (e.g., adders) increment counters for iterative processing.

2.2 Sequential Logic

Sequential logic elements like registers and finite state machines (FSMs) are employed to store intermediate values and control the flow of operations. Key components include:

- Registers store values such as current maximum/minimum, memory addresses, and counter values.
- FSMs manage the state transitions required for initialization, data processing, and output generation.

2.3 Integration of Modules

The circuit integrates three primary modules:

- Controller Module: Directs operations using FSM logic.
- Datapath Module: Handles data manipulation through combinational logic circuits.
- Memory Module: Stores input data for processing and retrieval.

This modular approach ensures efficient data flow and control, enabling accurate computation of maximum and minimum values while maintaining synchronization across all components.

By combining these elements, the circuit achieves a robust design capable of performing complex digital operations with precision.

3 ASM Chart

The ASM diagram represents a sequential process for calculating and outputting the minimum and maximum values from a dataset. It begins with initialization, followed by input handling, memory referencing, and iterative comparisons to determine the minimum and maximum values. Conditional branching guides the flow between states, ensuring accurate updates to variables holding these values. Once the calculations are complete, the results are outputted, and the process terminates. The diagram effectively combines memory operations, decision-making, and output handling in a structured manner.

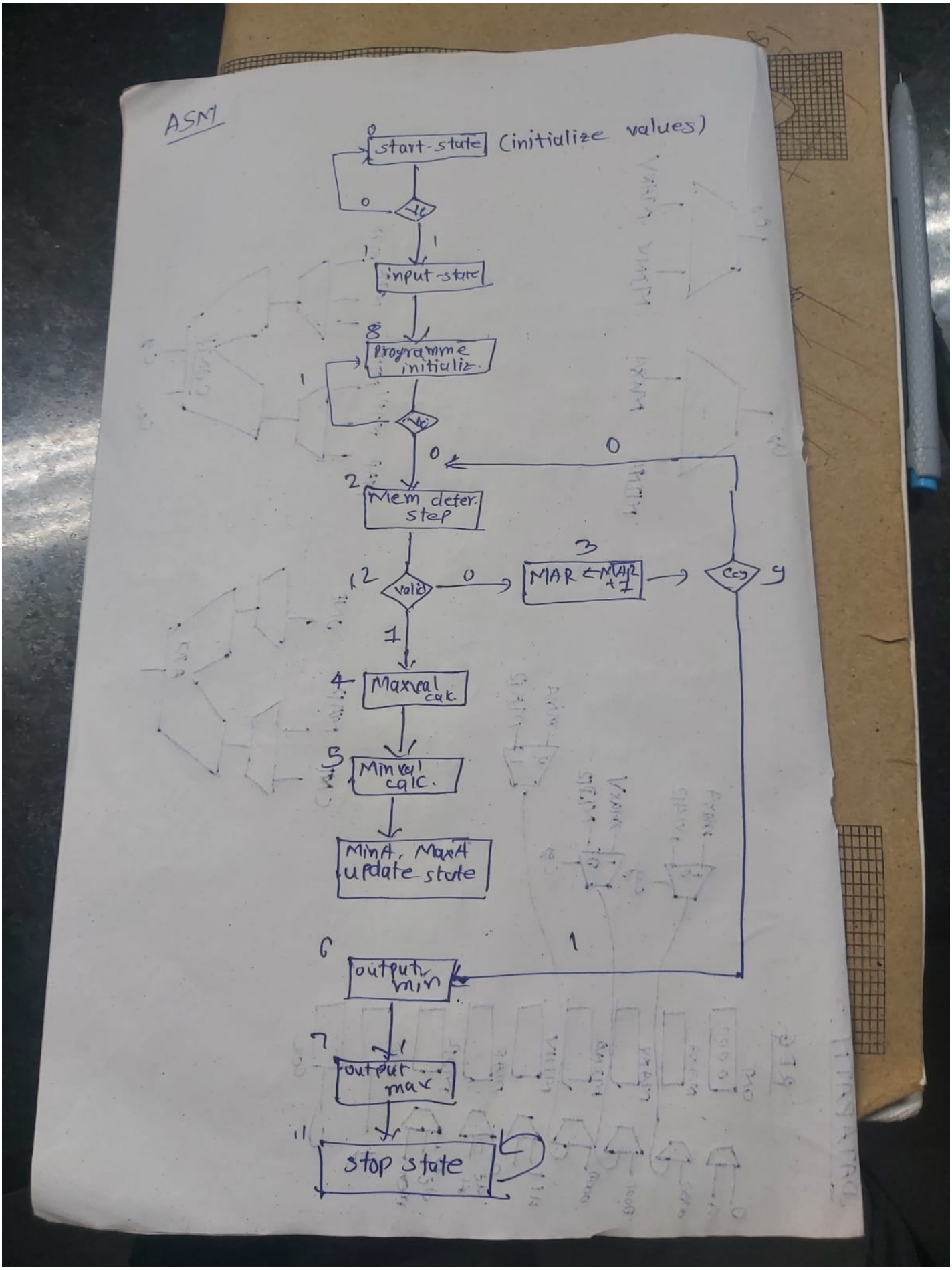

Figure 1: ASM Chart (Conceptual Representation)

4 Controller Section

4.1 FSM Chart

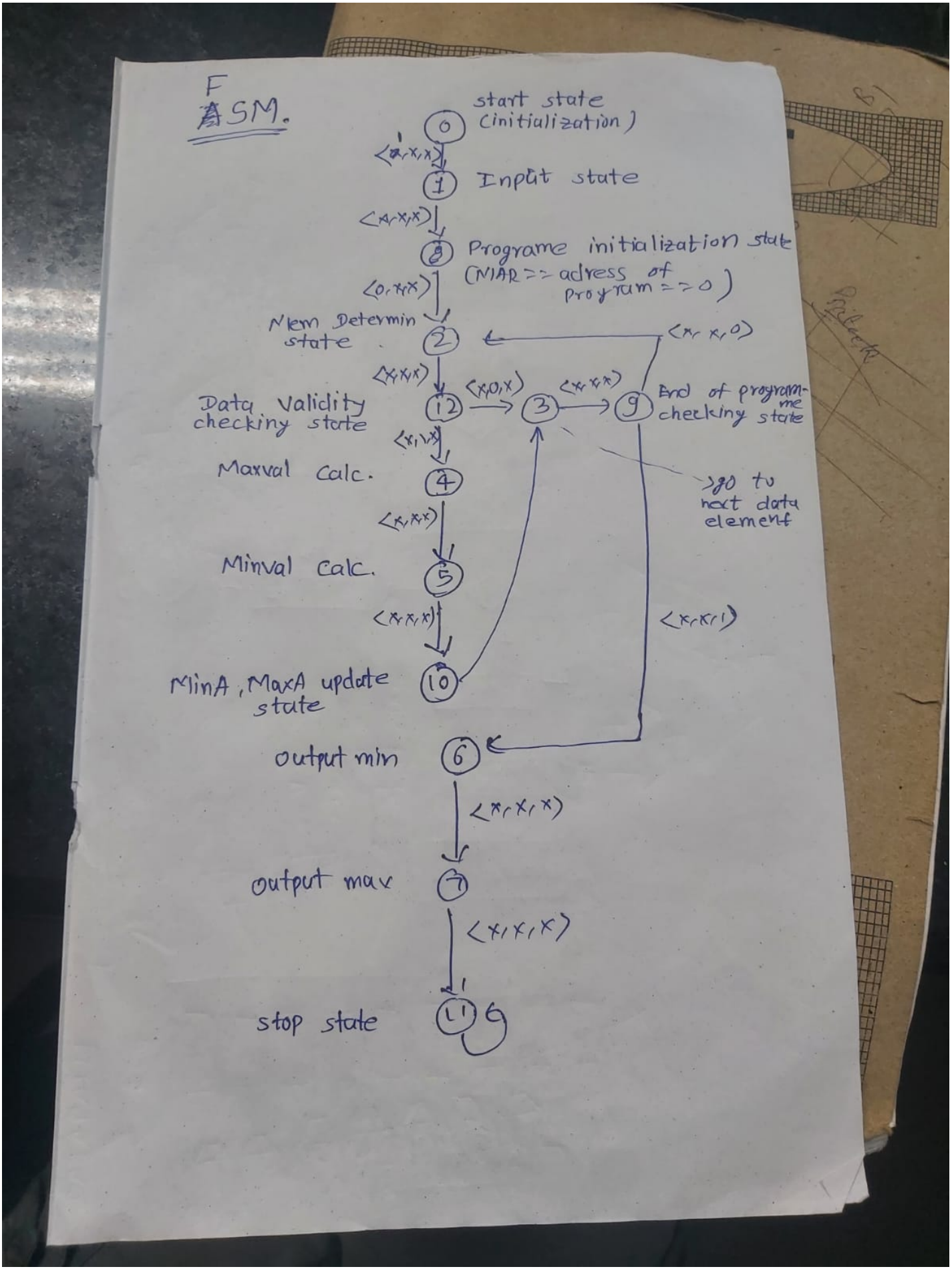

Figure 2: FSM Chart

5 Datapath Section

The datapath includes several registers: CNT (initialized to zero), MAXA and MAXV (maximum address and value), MINA and MINV (minimum address and value), MAR (Memory Address Register), MDR (Memory Data Register), and a constant register called ONE (value of 1). These registers are connected via multiplexers and control signals, which select inputs based on signals like CY (carry) and CE (control enable). The datapath features an adder (ADD) for arithmetic operations with inputs such as CNT and ONE, as well as a comparator (COMP) for comparing MAR, MAXV, and MINV. Outputs are represented by AO (address output) and DO (data output) sourced from the minimum and maximum registers.

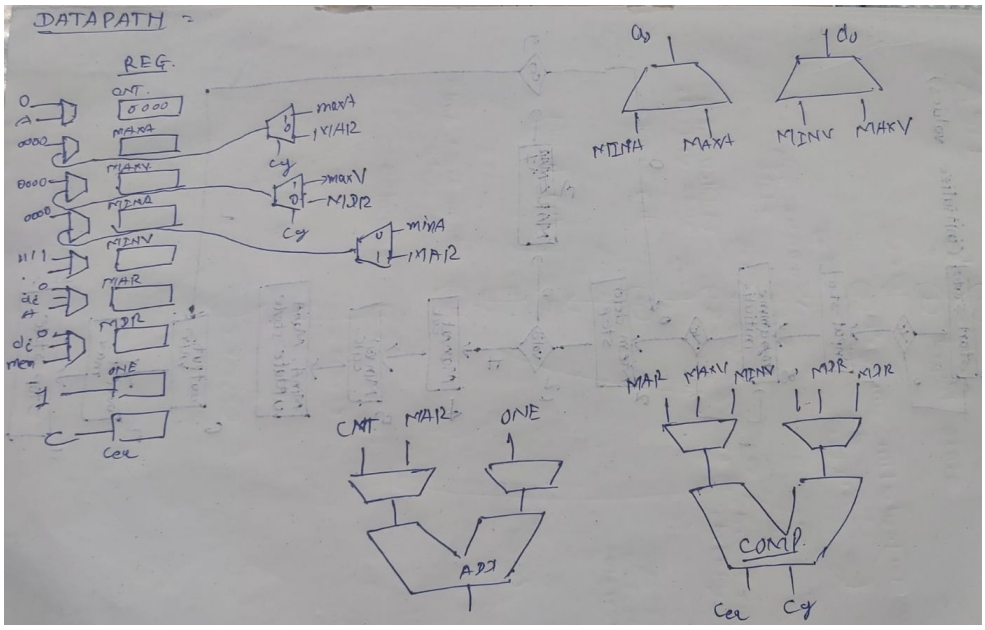

Figure 3: Datapath Diagram

6 Testbench

6.1 Simulation Results

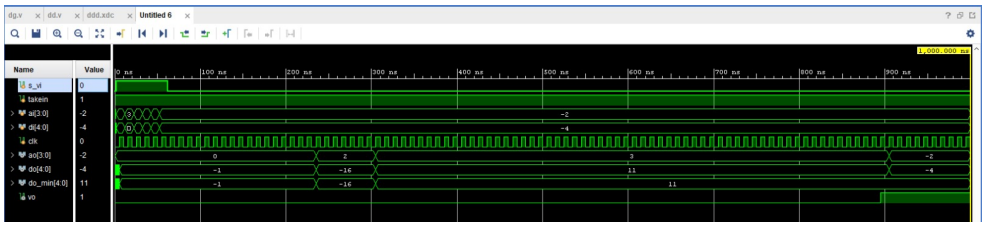

Figure 4: Simulation waveform in Vivado (Placeholder)

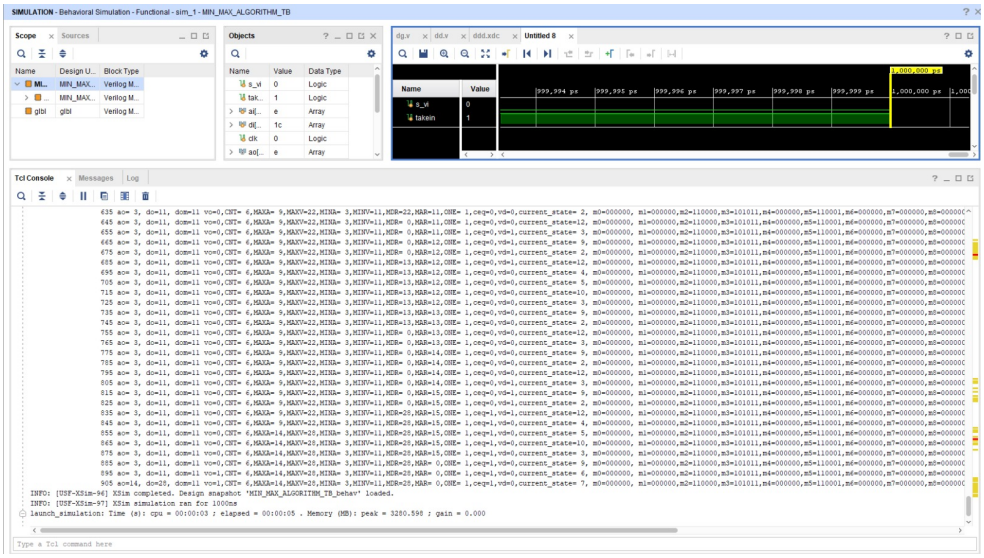

Figure 5: Simulation data in Vivado

7 I/O Pin Mapping

Table 1: I/O Pin Mapping

Signal Name	Port Name	FPGA Pin
s_vi	s_vi	V10
ai[3]	ai[3]	U11
ai[2]	ai[2]	U12
ai[1]	ai[1]	H6
ai[0]	ai[0]	T13
di[4]	di[4]	R16
di[3]	di[3]	U8
di[2]	di[2]	T8
di[1]	di[1]	R13
di[0]	di[0]	U18
ao[0]	ao[0]	H17
ao[1]	ao[1]	K15
ao[2]	ao[2]	J13
ao[3]	ao[3]	N14
takein	takein	T18
vo	vo	V11
do_min[4]	do_min[4]	V14
do_min[3]	do_min[3]	V15
do_min[2]	do_min[2]	T16
do_min[1]	do_min[1]	U14
do_min[0]	do_min[0]	T15
do[0]	do[0]	R18
do[1]	do[1]	V17
do[2]	do[2]	U17
do[3]	do[3]	U16
do[4]	do[4]	V16

8 Images

8.1 Block Diagrams

This image displays a schematic diagram of a digital system design in Vivado. It includes three main modules controller_module, memory_module, and datapath_module, interconnected with various signals. Inputs like ai[3:0], di[4:0], and control signals (takein, s_vi) are connected to the controller, while outputs such as ao[3:0], do[4:0], and vo are generated. The design uses multiplexers, registers, and memory elements to manage data flow and control logic.

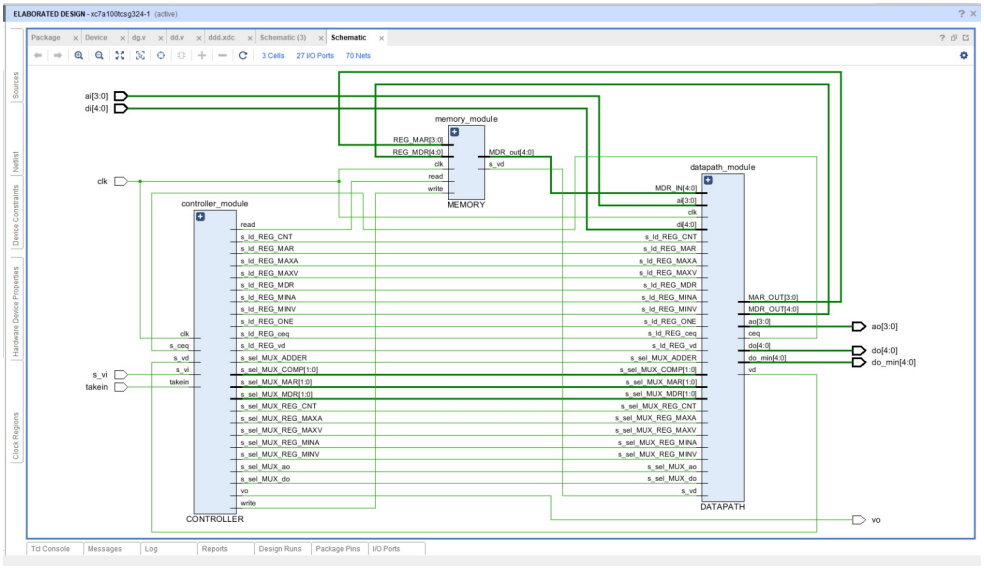

Figure 6: Block Diagram 1

This image represents a Vivado block design showcasing a digital system architecture. It includes three primary modules: controller_module, memory_module, and datapath_module, connected through a complex network of signals and nets. The design features 31 cells, 27 I/O ports, and 122 nets. Input signals such as ‘ai[3:0]‘, ‘di[4:0]‘, and control signals (‘takein‘, ‘s_vi‘) are routed into the modules, while outputs like ‘ao[3:0]‘, ‘do[4:0]‘, and ‘vo‘ are generated. The system uses buffers, multiplexers, and registers to manage data flow and control logic efficiently.

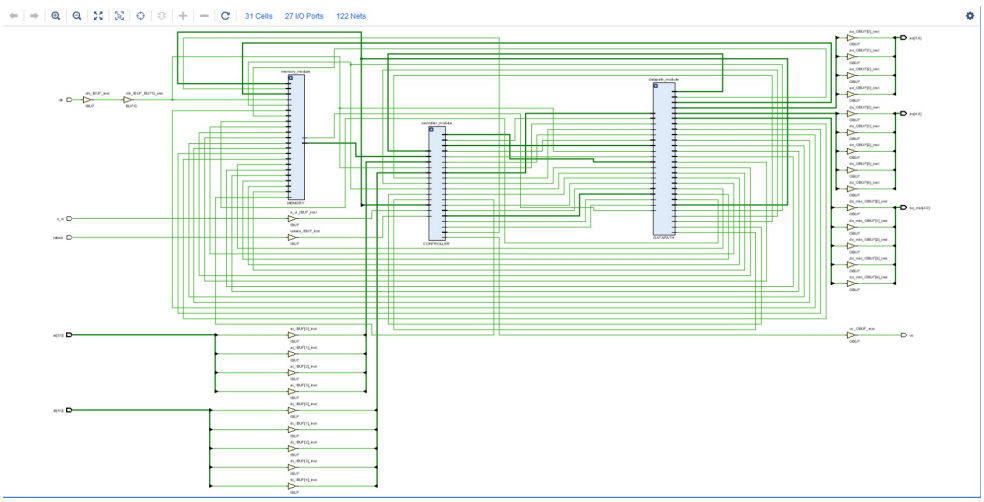

Figure 7: Block Diagram 2

9 Conclusion

In this report, we successfully implemented and analyzed a digital system design for calculating and outputting the maximum and minimum values from a dataset. The design incorporated a modular approach, including the controller, datapath, and memory sections, each contributing to efficient data processing and management. The ASM and FSM charts provided a clear visualization of the sequential logic and control flow, ensuring accurate operations.

The datapath design demonstrated the use of registers, multiplexers, comparators, and arithmetic units to perform iterative calculations and manage data flow effectively. Simulation results validated the functionality of the system, showcasing correct outputs for given inputs. Furthermore, the I/O pin mapping ensured seamless integration with FPGA hardware for real-world implementation.

Overall, this project highlights the importance of structured design methodologies in digital systems. By combining theoretical concepts with practical implementation, we achieved a robust solution to the problem statement. This work serves as a foundation for further enhancements and applications in hardware design.